Assignment 1

Fall 2026: COMP 480/580

Due Date: 10/21/2026

1 Introduction

Large Language Models (LLMs) are increasingly capable of processing long contexts. However, processing large amounts of text can be computationally expensive, with inference cost, input-token usage, and often latency increasing with context length. In this assignment, we will explore whether Bloom filters can be used to reduce the amount of context processed by an LLM while preserving question-answering accuracy. A Bloom filter is a space-efficient probabilistic data structure for testing whether an element belongs to a set. Bloom filters may produce false positives but do not produce false negatives for elements that were actually inserted, assuming that the filter is implemented correctly and elements are not deleted. Given N distinct inserted elements, a bit array of size R, and k hash functions, the false positive rate is approximately $p_{\text{fp}} = \left(1 - e^{-kN/R}\right)^k$, and the approximately optimal number of hash functions is

$$k = \frac{R}{N} \ln 2.$$

2 Bloom Filters for Context Compression

The contexts, corresponding questions and answers for this assignment are provided in the assignment repository: GitHub Link. Each context consists of a long piece of text and a set of questions that can be answered using information contained in the text. For each question, you will establish a **full-context baseline** by passing the entire context and the question to the LLM:

$$\text{Baseline:} \qquad \text{LLM}(\text{Full Context} + \text{Question}).$$

You can compare the answers given by the baseline with the answers that exist in the `answers` directory. Your goal is to design a Bloom Filter-based method that reduces the amount of context passed to the LLM. How you use the Bloom filter is intentionally left open-ended. For each question, your method should produce a compressed context that is then passed to the same LLM:

$$\text{Your Method:} \qquad \text{Full Context} \xrightarrow{\text{Bloom Filter}} \text{Compressed Context} \xrightarrow{+\text{ Question}} \text{LLM}.$$

Your method will be evaluated on the **same contexts, questions, model, and prompting setup** as the full-context baseline. In addition to the provided contexts and questions, we will evaluate your implementation on **additional held-out test contexts and questions** that will not be released. Your implementation should therefore be general and should not rely on context-specific rules, hard-coded answers, or assumptions specific to the provided datasets.

Your objective is to improve efficiency by reducing **LLM input token usage and latency**, while preserving or improving **question-answering accuracy**. You should also measure the **memory usage** of your Bloom-Filter based method.

Use `gemini-3.5-flash-lite` for all experiments. Keep the model and prompting strategy fixed across all experiments so that improvements can be attributed to your Bloom-filter based context reduction method.

2.1 Contexts and Questions

The repository contains eight contexts, organized into four `v1` and four `v2` contexts. For this assignment, you are required to use only the four `v1` contexts in the `contexts` directory:

- `antikythera_dossier_v1.txt`
- `dwh_dossier_v1.txt`
- `harbour_puzzle_v1.txt`
- `lunarleaper_2609_11453.txt`

The `v1` contexts are approximately 15K-50K tokens long. The corresponding `v2` contexts are significantly longer and may require more capable and potentially more expensive LLMs to process effectively. You are welcome to experiment with the `v2` contexts, but this is **not required** for the assignment. Each `v1` context has exactly 20 associated questions. You will therefore answer exactly **80 questions in total** located in `questions` directory. For each question, your system receives the corresponding full context and question. For example, an evaluation instance may be:

`<antikythera_dossier_v1.txt>` + one question.

Your Bloom-filter-based method should reduce this context. Your performance will be compared against the provided full-context baseline using the following metrics:

- **Accuracy:** Fraction of questions answered correctly by the LLM, evaluated against the `expected` answers and `aliases` in the provided `answers` directory, using the specified numerical tolerances and units.

- **Token Usage:** Total number of input tokens passed to the LLM, including the retained context, question, and any additional prompt.
- **Latency:** Total time required to retrieve the relevant context using your Bloom Filter-based method and generate the answer using the LLM.
- **Memory Usage:** Total memory required by the Bloom filter(s) and any additional data structures you use.

The goal is **not simply to minimize context length**. An effective solution should reduce token usage and latency while maintaining accuracy as close as possible to the full-context baseline. You should experimentally study and report this tradeoff as you vary the parameters of your Bloom-Filter. Your implementation will also be evaluated for reproducibility. We will run your submitted code using your Bloom-Filter implementation and verify the reported accuracy, token usage, latency, and memory usage. Therefore, make sure that your experiments can be reproduced without manual intervention.

3 Tasks

1. **Bloom Filter implementation:** Implement your own Bloom-Filter supporting insertion and membership queries. You may use an existing hash function implementation, but you should not use an existing Bloom filter library. Get creative at this step. Each solution can be different, and we are looking for different efficient solutions.
2. **Vary the Bloom Filter parameters:** Repeat the experiment for different Bloom filter sizes R and numbers of hash functions k. Study how these parameters affect:
    - False Positive Rate
    - Bloom filter Memory Usage
    - Amount of Context Retained
    - LLM Input Token Usage
    - Question-Answering Accuracy
3. **Plots:** Create plots showing the important tradeoffs you observe. At minimum, include:
    - Bloom Filter Memory vs. False Positive Rate
    - Bloom Filter Memory vs. LLM input Tokens
    - LLM input tokens vs. Question-Answering Accuracy.

4 Questions

Answer the following questions in your report.

1. How does the measured false positive rate compare with the theoretical false positive rate of your Bloom filter?
2. How do R and k affect the false positive rate and memory requirements of the Bloom filter?
3. How do Bloom filter false positives affect the compressed context, input-token usage, and question-answering accuracy?
4. How much can you reduce the number of input tokens while maintaining accuracy close to the full-context baseline?
5. Is using a larger Bloom filter always beneficial? Discuss the tradeoff between Bloom filter memory, token usage, and accuracy.
6. What are the limitations of using Bloom filters for compressing context?

5 Grading (100 Points)

- **Bloom Filter Implementation: 20 points**
- **Bloom-Filter-Based Context Compression Method: 20 points**
- **Experimental Evaluation and Parameter Study: 25 points**
- **Required Plots: 10 points**
- **Report and Answers to the Questions: 15 points**
- **Code Quality and Reproducibility: 10 points**

6 What to Submit

Submit your source code (ideally, this should not be more than 2 python files), a `requirements.txt` file, a README containing the exact execution command, and a short report describing your implementation, results, and generated plots. Your code must reproduce all reported experiments, and the TAs will rigorously verify its reproducibility. Fix all random seeds whenever applicable to ensure consistent results across runs.